Submitted by:Vismaya T M

MindMatrix Internship-Android App Development using Gen AI

Deep Dive into UI State

Introduction:

Interactive buttons are fundamental components in modern Android applications because they allow users to perform actions such as submitting input, triggering logic, or navigating between screens. In Jetpack Compose, button interaction follows a reactive programming model where a user action leads to a change in UI state, and the system automatically updates the interface through recomposition.

UI state refers to the data that determines what is displayed on the screen at any given time. In Compose, UI state is typically stored using remember and mutableStateOf. When this state changes, Compose automatically recomposes only the parts of the UI that depend on that state. This makes the UI dynamic, efficient, and easier to manage compared to traditional imperative UI approaches.

User Action → onClick Handler → State Update → UI Recomposition. I analyse button interaction using two apps I built:

- **Dice Roller App**
- **Lemonade App**

Both apps demonstrate how Compose manages UI state and recomposition efficiently when buttons are clicked.

Task 1 — Button Interaction Fundamentals & Analysis

1 Major Learning Goals (Add a Button to an App):

- Respond to user taps
- Use Button composable
- Handle onClick events
- Update state using remember
- Trigger recomposition automatically
- Display updated UI using Text or Image

2 Sample Compose Code (Dice Roller as an Example):

```
@Composable
fun DiceRollerApp() {
```

```
    var result by remember { mutableStateOf(1) }

    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center,
        modifier = Modifier.fillMaxSize()
    ) {

        Image(
            painter = painterResource(id = getDiceImage(result)),
            contentDescription = "Dice Image"
        )

        Spacer(modifier = Modifier.height(16.dp))

        Button(
            onClick = {
                result = (1..6).random()
            }
        ) {
            Text("Roll")
        }
    }
}
```

Annotation:

- **Where is the button defined?**

```
Button(
    onClick = { ... }
)
```

- **How does click handler modify state?**

result = (1..6).random()

This changes the value of result.

- **Which UI recomposes?**
  - The Image() composable recomposes
  - Because it depends on result
  - When result changes → image updates

4️⃣ How Compose Handles Recomposition:

Jetpack Compose uses a reactive state model. When a state variable created using remember { mutableStateOf() } changes, Compose automatically recomposes only the composables that depend on that state. In the Dice app, when the button updates result, only the Image composable redraws. This makes UI updates efficient and automatic.

5️⃣ Reflection — Challenges in Interactive UI:

1. Managing multiple states without confusion.
2. Avoiding unnecessary recompositions.
3. Preventing fast repeated clicks causing bugs.

To solve this, I will:

- Use proper state modelling.
- Separate UI and logic.
- Disable buttons when needed.
- Test edge cases like rapid clicks.

Task 2 — Multi-Button Interactive Design

Feature Idea: Dice App Extended Version

Buttons:

- Roll Dice
- Reset Dice

Layout Sketch (Text Representation):

[Dice Image]

[Roll Button]
[Reset Button]

[Result Text]

Button Actions & State Changes:

- **Button A — Roll**

- Action: Generate random number
- State Change: result updates
- UI Feedback: Dice image changes

◆ **Button B — Reset**

- Action: Reset result to 1
- State Change: result = 1
- UI Feedback: Dice shows default image

State Table:

Trigger Button	Resulting State	UI Change
Roll	result = random(1..6)	Dice image changes
Reset	result = 1	Dice resets to default
Roll repeatedly	result changes every click	Continuous image update

Reflection:

If state is not managed cleanly, UI may show wrong images or stale values. Without proper state handling, reset may not work correctly. I will test state transitions by clicking buttons repeatedly and rotating screen to ensure state consistency.

Task 3 — Usability & Accessibility Review

Using my **Lemonade App**

Review Table:

Issue	Description	Proposed Fix	Priority
Touch Target	Image click area slightly small	Ensure 48dp minimum	High
Accessibility	contentDescription needed	Add meaningful description	High
Contrast	Light background + light text	Increase contrast	Medium
Orientation	UI may stretch	Use responsive layout	Medium

Mini Report:

Usability and accessibility ensure apps are inclusive and easy to use. Buttons must be large enough and provide visual feedback. Accessibility features like contentDescription help screen readers. Good button design improves user satisfaction and reduces errors.

Task 4 — Interactive Feature Plan

Feature: Advanced Lemonade App with Reset

Feature Description

An interactive app where:

- User taps tree → pick lemon
- Tap lemon → squeeze
- Tap empty glass → restart
- Reset button → return to initial state

Audience: Beginner Android users learning state handling.

Button List & State Model

States:

1. Select
2. Squeeze
3. Drink
4. Restart

State Table:

Current State	Button Action	Next State
Select	Tap tree	Squeeze
Squeeze	Tap lemon	Drink
Drink	Tap glass	Restart
Restart	Tap	Select

Pseudocode:

```
state = "select"

onImageClick:
    if state == "select":
        state = "squeeze"
    else if state == "squeeze":
        state = "drink"
    else if state == "drink":
        state = "restart"
    else:
        state = "select"
```

Advanced Case:

Disable button when squeezing count is finished.

Test Plan:

1. Tap through full cycle → Should return to start.
2. Rotate screen → State should remain.
3. Rapid tapping → Should not skip states incorrectly.

Reflection:

Challenges include managing multiple states and preventing incorrect transitions. Fast clicks may cause bugs. I will handle this using proper state variables and conditional logic. Testing orientation changes and repeated clicks is important to ensure stability.

🔷 Conclusion:

Through the Dice Roller and Lemonade apps, I understood the complete interaction flow in Jetpack Compose:

User Click → onClick Handler → State Change → Recomposition → UI Update State management is the foundation of interactive UI. Proper modelling of states prevents inconsistent UI behaviour. By analysing button interaction, designing multi-button screens, reviewing usability, and planning advanced features, I developed a strong understanding of interactive Compose applications.This structured planning approach will help me build scalable, maintainable, and user-friendly Android applications in the future.

Submitted by: Vismaya T M

MindMatrix Internship-Android App Development using Gen AI